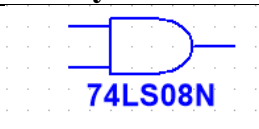
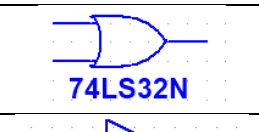
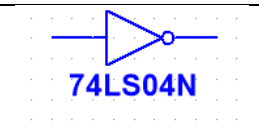
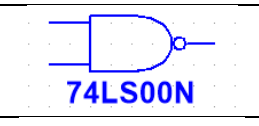
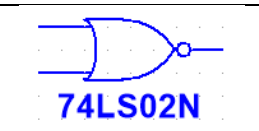
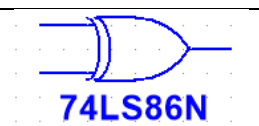
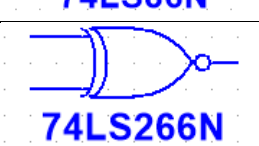
LAB-1

Part-A) VERIFICATION OF LOGIC GATES

Aim: To verify the logic gates

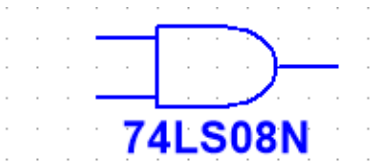
List of Logic gates:

Name	Symbol	IC No
AND		74LS08N
OR		74LS32N
NOT		74LS04N
NAND		74LS00N
NOR		74LS02N
XOR		74LS86N
XNOR		74LS266N

<u>AND GATE:</u>

Description: If two inputs are 1 then AND gate output is 1 otherwise 0.

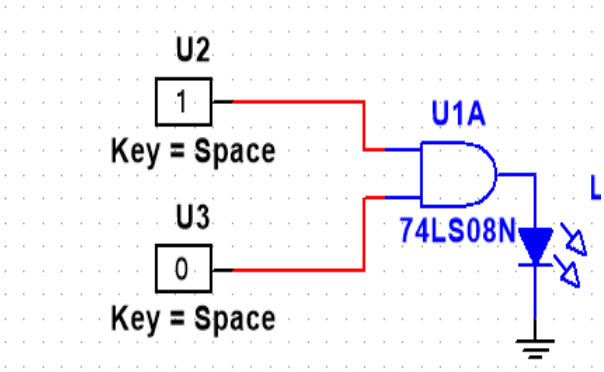
Circuit Diagram:



Truth Table:

A	B	Y=A.B
0	0	0
0	1	0
1	0	0
1	1	1

Multisim Circuit:



Components required:

1.Interactive digital constants

2.And gate

3.LED

4.Ground

Procedure:

1.Open multisim software and create a new schematic.

2.Place AND gate, 2 Interactive digital constants, LED and Ground on the schematic.

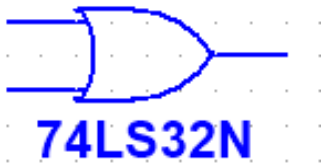
3.Connect the circuit as per circuit diagram.

4.Now verify the circuit.

OR GATE:

Description: If one of the input is 1 then OR gate output is 1 otherwise 0.

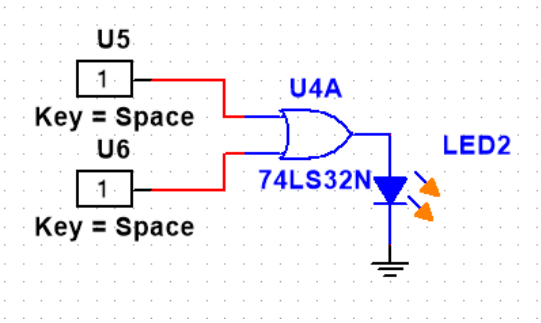
Circuit Diagram:



Truth Table:

A	B	Y=A+B
0	0	0
0	1	1
1	0	1
1	1	1

Multisim Circuit:



Components required:

1.Interactive digital constants

2.OR gate

3.LED

4.Ground

Procedure:

1.Open multisim software and create a new schematic.

2.Place OR gate, 2 Interactive digital constants, LED and Ground on the schematic.

3.Connect the circuit as per circuit diagram.

4.Now verify the circuit.

NOT gate:

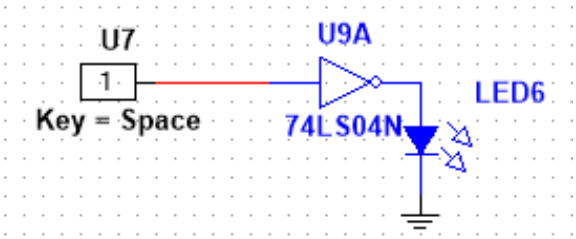
Description: If the input is 1 then its output will be 0 and its vice-versa.

Circuit diagram:

Truth table:

A	A’
0	1
1	0

Multisim circuit:



Components required:

1.Interactive digital constants

2.NOT gate

3.LED

4.Ground

Procedure:

1.Open multisim software and create a new schematic.

2.Place NOT gate, 1 Interactive digital constants, LED and Ground on the schematic.

3.Connect the circuit as per circuit diagram.

4.Now verify the circuit.

NAND GATE:

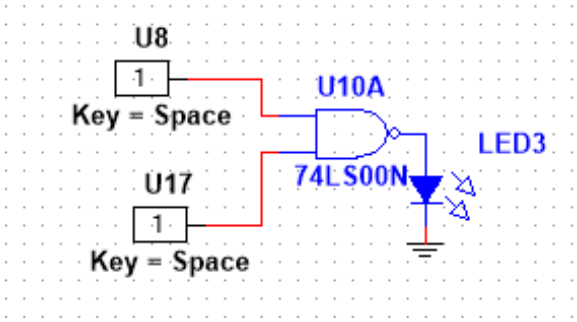
Description: If two inputs are 1 then NAND gate output is 0 otherwise 1. It is not gate to the AND gate.

Circuit Diagram:

Truth Table:

A	B	Y=A.B
0	0	0
0	1	0
1	0	0
1	1	1

Multisim Circuit:



Components required:

1.Interactive digital constants

2.NAND gate

3.LED

4.Ground

Procedure:

1.Open multisim software and create a new schematic.

2.Place NAND gate, 2 Interactive digital constants, LED and ground on the schematic.

3.Connect the circuit as per circuit diagram.

4.Now verify the circuit.

NOR GATE:

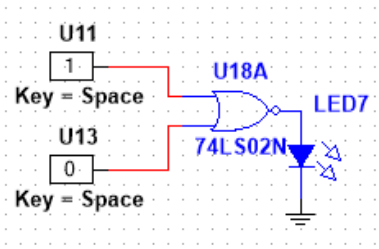
Description: If any one of the input is 1 then NOR gate output is 0 otherwise 1. It is not gate to the AND gate.

Circuit Diagram:

Truth Table:

A	B	Y=A.B
0	0	1
0	1	0
1	0	0
1	1	0

Multisim Circuit:



Components required:

1.Interactive digital constants

2.NOR gate

3.LED

4.Ground

Procedure:

1.Open multisim software and create a new schematic.

2.Place NOR gate, 2 Interactive digital constants, LED and Ground on the schematic

3.Connect the circuit as per circuit diagram.

4.Now verify the circuit.

NAND GATE:

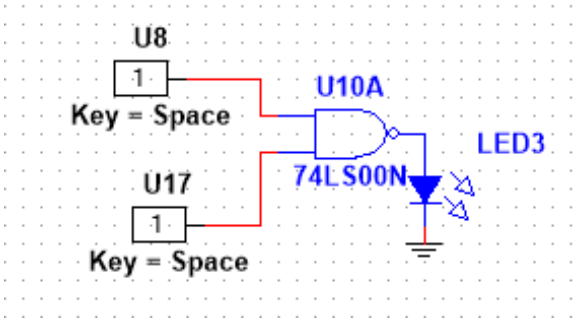
Description: If two inputs are 1 then NAND gate output is 0 otherwise 1. It is not gate to the AND gate.

Circuit Diagram:

Truth Table:

A	B	Y=A.B
0	0	0
0	1	0
1	0	0
1	1	1

Multisim Circuit:



Components required:

1.Interactive digital constants

2.NAND gate

3.LED

4.Ground

Procedure:

1.Open multisim software and create a new schematic.

2.Place NAND gate, 2 Interactive digital constants, LED and Ground on the schematic.

3.Connect the circuit as per circuit diagram.

4.Now verify the circuit.

XOR GATE:

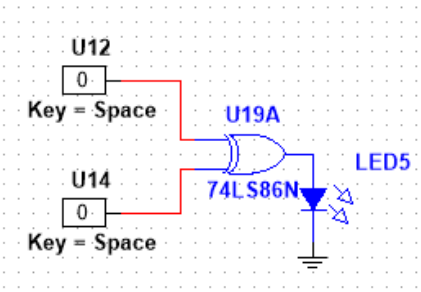
Description: If both the input are same then XOR gate output is 0 otherwise 1.

Circuit Diagram:

Truth Table:

A	B	Y=A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Multisim Circuit:



Components required:

1.Interactive digital constants

2.XOR gate

3.LED

4.Ground

Procedure:

1.Open multisim software and create a new schematic.

2.Place XOR gate, 2 Interactive digital constants, LED and Ground on the schematic.

3.Connect the circuit as per circuit diagram.

4.Now verify the circuit.

XNOR GATE:

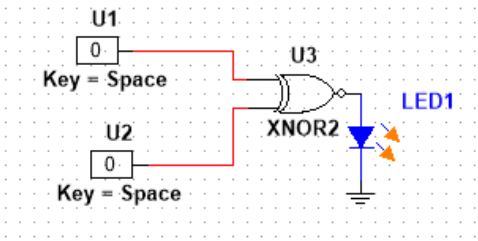
Description: If both the input are same then XNOR gate output is 1 otherwise 0. It is opposite to the XOR gate.

Circuit Diagram:

Truth Table:

A	B	Y=A XNOR B
0	0	1
0	1	0
1	0	0
1	1	1

Multisim Circuit:



Components required:

1. Interactive digital constants
2. XNOR gate
3. LED
4. Ground

Procedure:

1. Open multisim software and create a new schematic.
2. Place XNOR gate, 2 Interactive digital constants, LED and Ground on the schematic.
3. Connect the circuit as per circuit diagram.
4. Now verify the circuit.

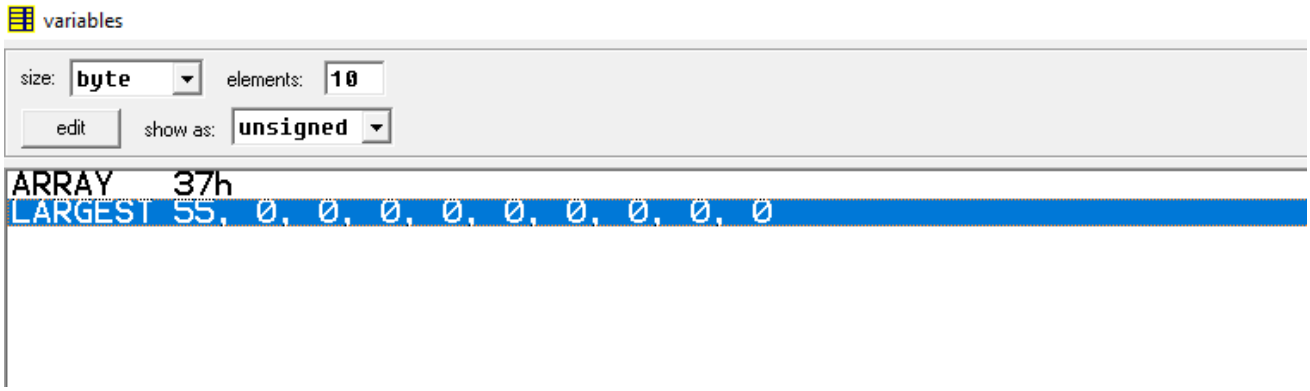
Part-B)

Aim: Assembly language program to find largest number in Array**.**

Input code:

```
.MODEL SMALL
.STACK 1001
.DATA
ARRAY DB 55,32,8,10
LARGEST DB ?
.CODE
MAIN PROC
   MOV AX,@DATA
   MOV DS,AX,BX
   MOV AL,ARRAY[BX]
   MOV LARGEST,AL
   COMPARE:
   INC BX
   CMP BX,1
   JE EXIT
   MOV AL,ARRAY[BX]
   CMP AL,LARGEST
   JG UPDATE_LARGEST
   JMP COMPARE
   UPDATE_LARGEST:
   MOV LARGEST,AL
   JMP COMPARE
   EXIT:
   MAIN ENDP
END MAIN
```

Output:


variables
size: byte
elements: 10
edit
show as: unsigned
ARRAY 37h
LARGEST 55, 0, 0, 0, 0, 0, 0, 0, 0, 0

LAB-2

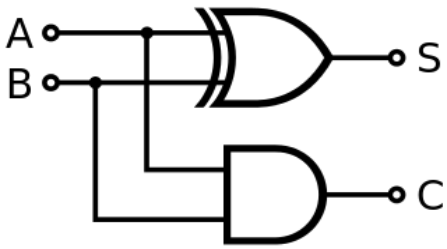
Part-A: ADDERS AND SUBTRACTORS

Aim: To verify adders and subtractors

Half Adder:

Description: A logic gate for the addition of two one bit numbers is referred as Half Adder. It has two inputs A(Augend) and B(Addend) and produces two outputs one for the sum and other for carry.

Circuit Diagram:



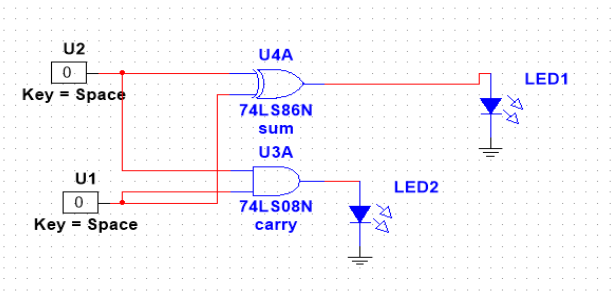
Truth Table:

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Components Required:

- Interactive digital constants
- XOR gate
- AND gate
- LED
- Ground

Multisim Circuit:



Procedure:

1. Open multisim software create a new schematic.
2. Place 2 interactive digital constants,XOR gate,AND gate,2 LEDS and 2 Grounds on the schematic.

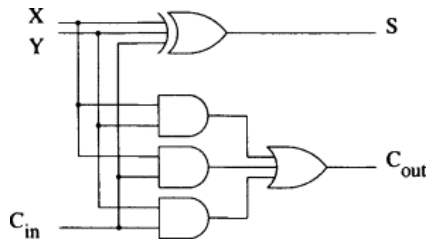
3. Connect the circuit as per circuit diagram.
4. Verify the circuit for all combinations.

Full Adder:

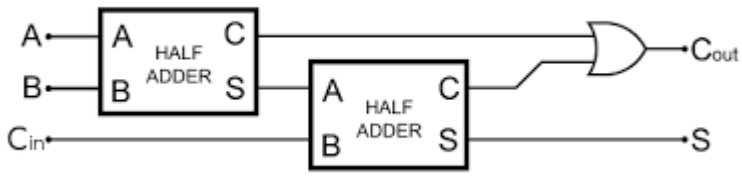
Description: Full Adder is the adder that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-IN. The output carry is designated as C-OUT and the normal output is designated as S which is SUM.

Circuit Diagram:

1)Using gates:



2)Using half adder:



Truth Table:

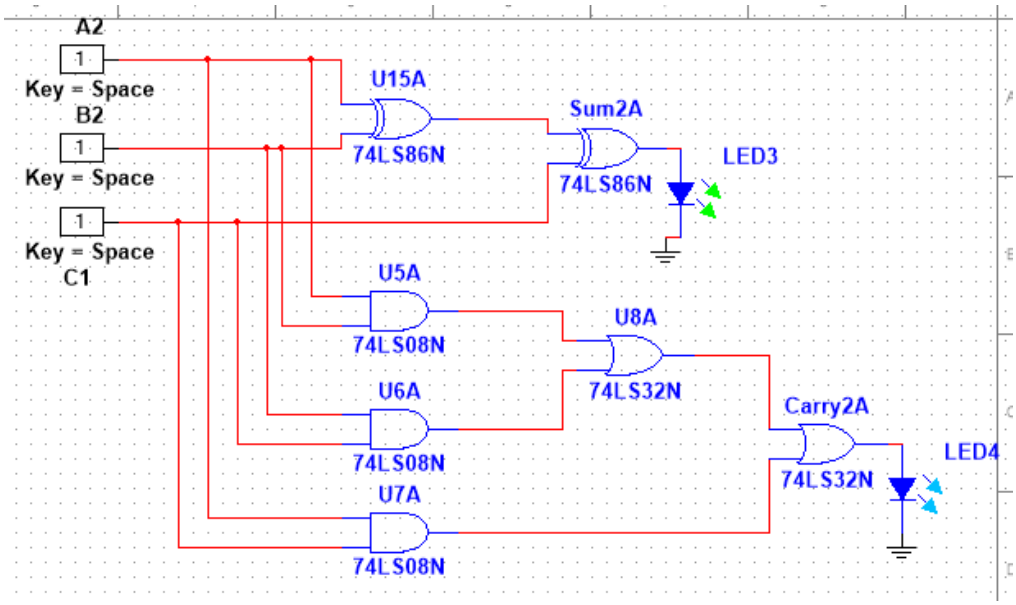
A	B	C	SUM	CARRY
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Components required:

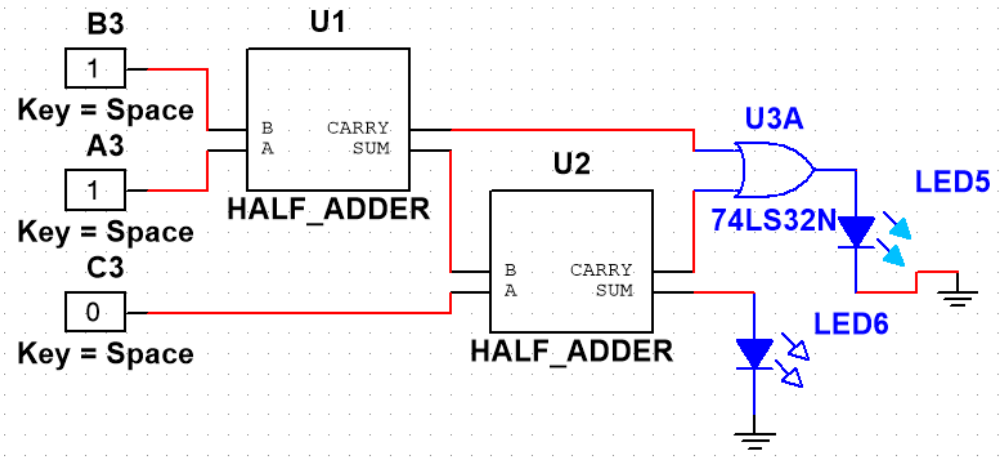
- Interactive digital constants
- XOR gate, AND gate, OR gate
- LED
- Ground

Multisim Circuit:

1)Using gates:



2.Using Half adder:



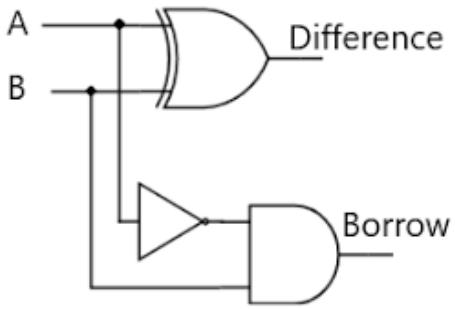
Procedure:

1. Open Multisim software and create new schematic.
2. Place XOR gate for sum.
3. For carry the equation is AB+BC+AC. Connect the circuit as per the equation.
4. Connect LED's to the outputs.
5. Connect ground to LED's and verify the circuit.
6. Full adder can also be constructed by using half adders.

Half Subtractor:

Description: This circuit is used to subtract two single bit binary numbers. It has two inputs and two outputs. The **difference** and **borrow** are two output states of the half subtractor.

Circuit Diagram:



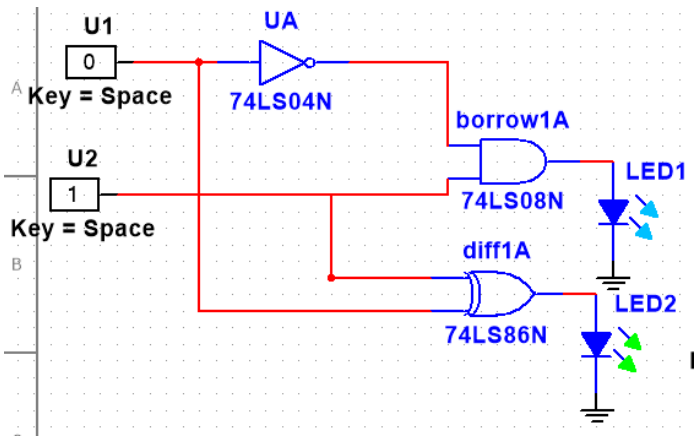
Truth Table:

A	B	DIFFERENCE	BORROW
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Components required:

- Interactive digital constants
- XOR gate, AND gate, NOT gate
- LED
- Ground

Multisim Circuit:



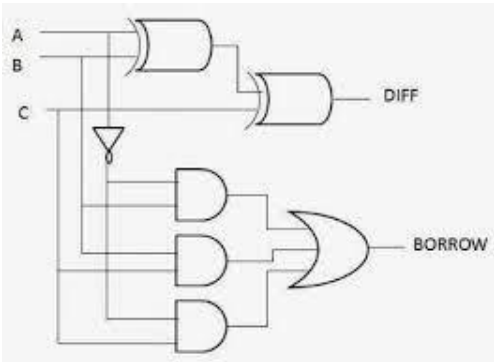
Procedure:

1. Open Multisim software and create new schematic.
2. Place XOR gate for difference, AND gate for borrow.
3. For difference the two interactive digital constants are connected to XOR gate.
4. For borrow one input is complemented using NOT gate and the two inputs are connected to AND gate.
5. Connect LED's to the output of XOR gate, AND gate.
6. Connect ground to LED's and verify the circuit.

Full Subtractor:

Description: The full subtractor is used to subtract three 1-bit numbers. The full subtractor has three input states and two output states i.e., difference and borrow.

Circuit Diagram:



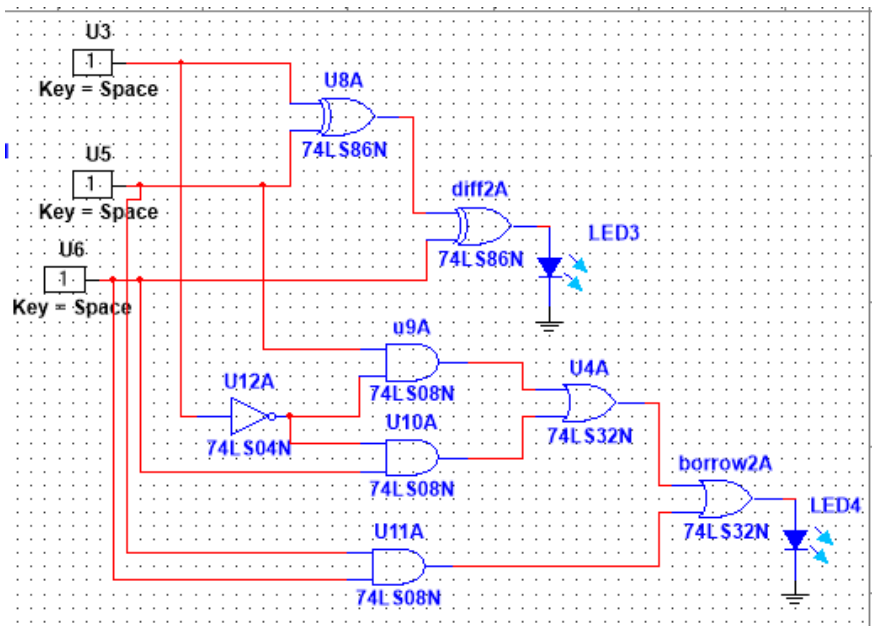
Truth Table:

A	B	C	DIFFERENCE	BORROW
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Components required:

- XOR gate, AND gate, NOT gate
- LED
- Ground
- Interactive digital constants

Multisim Circuit:



Procedure:

1. Open Multisim software and create new schematic.
2. Place XOR gate for difference.
3. For difference the equation is A'B+BC+A'C. Connect the circuit as per the equation.
4. Connect LED's to the outputs.
5. Connect ground to LED's and verify the circuit.
6. Full subtractor can also be constructed by using half subtractors.

Part-B:

Aim: Assembly language program to find smallest number in an Array.

Input code:

```
org 100h
include 'emu8086.inc'
.model small
.stack 100h
.data
array db 7,3,4,6,5
.code
main proc
   mov ax,@data
   mov ds,ax
   mov si,offset array
   mov cx,5
   mov bl,[si]
   Loopx:
   cmp [si],bl
   jle update
   resume:
   inc si
   loop loopx
   print "smallest value from array:"
   add bl,48
   mov dl,bl
   mov ah,02h
   int 21h
   update:
   mov bl,[si]
   jmp resume
   main endp
```

end main

Ret

Output:

emulator screen (113x41 chars)

```
smallest value from array:3
```

LAB-3

Part-A: RIPPLE CARRY ADDER AND CARRY LOOK AHEAD ADDER

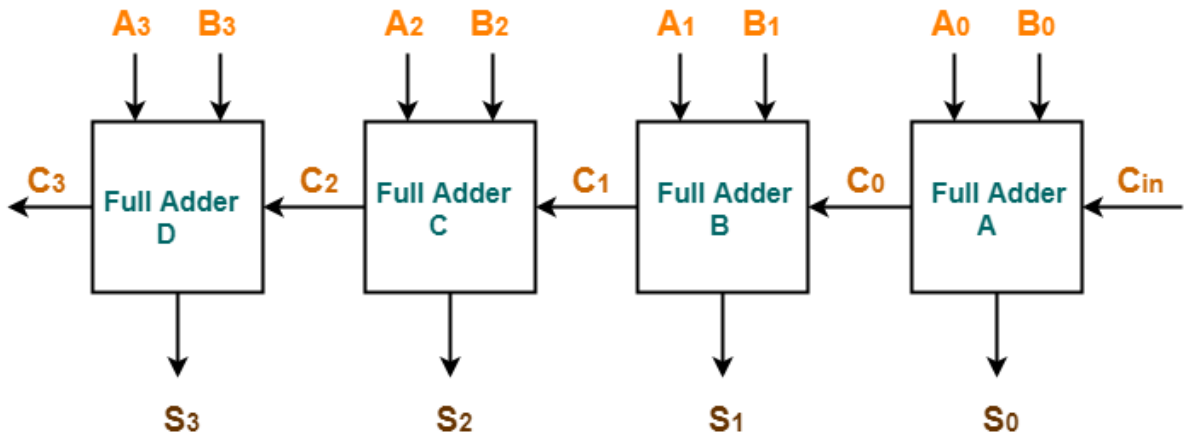
Aim: To verify ripple carry adder and carry look ahead adder.

Ripple Carry Adder:

Description:

- The ripple carry adder is constructed by cascading full adder block in series.
- The carryout of one stage is fed directly to carry in of next stage.
- This adder is ripple carry adder because each carry bit rippled into the next state.

Circuit Diagram:



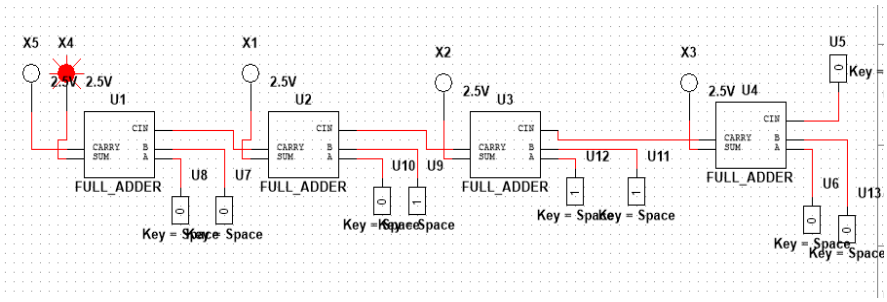
Truth table:

	A				B				Sum				Carry
Cin	A3	A2	A1	A0	B3	B2	B1	B0	S3	S2	S1	S0	Cout
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1	0	0	1	0	0
0	0	0	1	0	0	0	1	0	0	1	0	0	0
0	0	0	1	1	0	0	1	1	0	1	1	0	0
0	0	1	0	0	0	1	0	0	1	0	0	0	0
0	0	1	0	1	0	1	0	1	1	0	1	0	0
0	0	1	1	0	0	1	1	0	1	1	0	0	0
0	0	1	1	1	0	1	1	1	1	1	1	0	0
0	1	0	0	0	1	0	0	0	0	0	0	0	1
0	1	0	0	1	1	0	0	1	0	0	1	0	1
0	1	0	1	0	1	0	1	0	0	1	0	0	1
0	1	0	1	1	1	0	1	1	0	1	1	0	1
0	1	1	0	0	1	1	0	0	1	0	0	0	1
0	1	1	0	1	1	1	0	1	1	0	1	0	1
0	1	1	1	0	1	1	1	0	1	1	0	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

Components required:

- Interactive digital constants
- Full adders
- Probes

Multisim circuit:



Procedure:

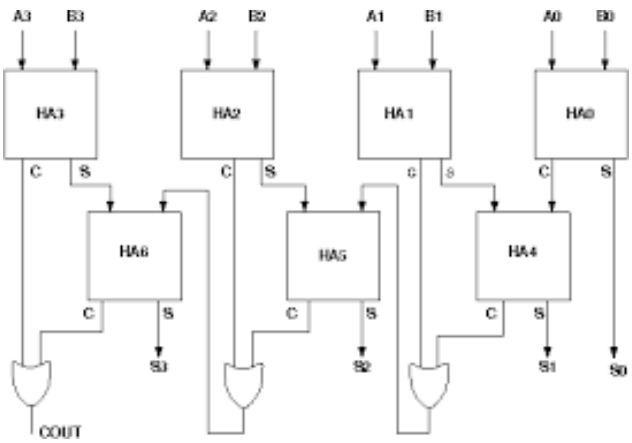
1. Open multisim software and create a new schematic.
2. Place 4 full adders,8 interactive digital constants and probes on the schematic.
3. Connect the circuit as per circuit diagram.
4. Verify the circuit for all combinations.

Carry-Look-Ahead-Adder:

Description:

- To reduce the computation time, there are faster ways to add two binary numbers by using carry look ahead adders.
- They work by creating two signals P and G known to be Carry Propagator and Carry Generator.
- The carry propagator is propagated to the next level whereas the carry generator is used to generate the output carry, regardless of input carry.

Circuit Diagram:



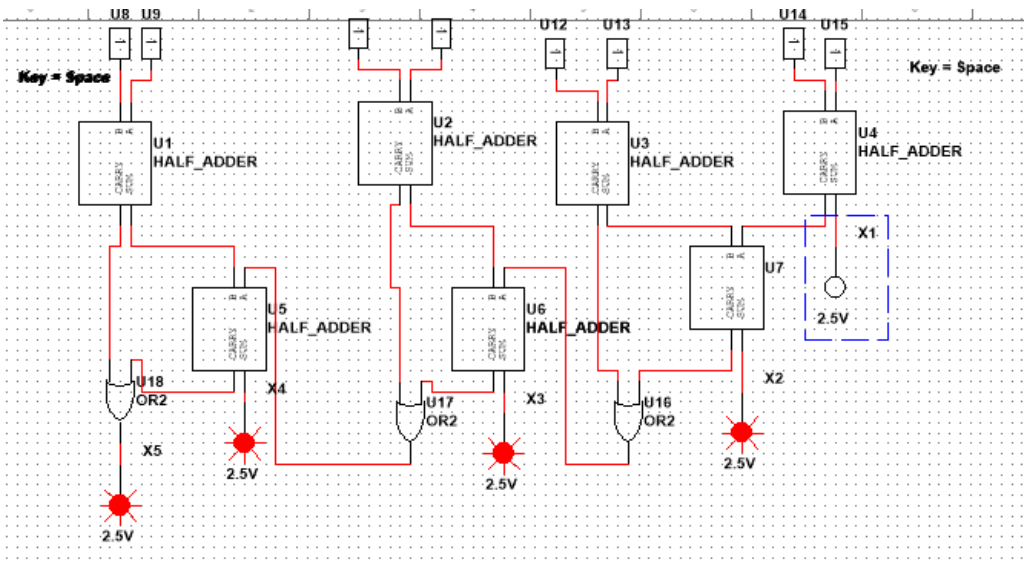
Truth Table:

	A				B				Sum				Carry
Cin	A3	A2	A1	A0	B3	B2	B1	B0	S3	S2	S1	S0	Cout
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1	0	0	1	0	0
0	0	0	1	0	0	0	1	0	0	1	0	0	0
0	0	0	1	1	0	0	1	1	0	1	1	0	0
0	0	1	0	0	0	1	0	0	1	0	0	0	0
0	0	1	0	1	0	1	0	1	1	0	1	0	0
0	0	1	1	0	0	1	1	0	1	1	0	0	0
0	0	1	1	1	0	1	1	1	1	1	1	0	0
0	1	0	0	0	1	0	0	0	0	0	0	0	1
0	1	0	0	1	1	0	0	1	0	0	1	0	1
0	1	0	1	0	1	0	1	0	0	1	0	0	1
0	1	0	1	1	1	0	1	1	0	1	1	0	1
0	1	1	0	0	1	1	0	0	1	0	0	0	1
0	1	1	0	1	1	1	0	1	1	0	1	0	1
0	1	1	1	0	1	1	1	0	1	1	0	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

Components required:

- Interactive digital constant
- Half adder
- Probe
- OR gate

Multisim Circuit:



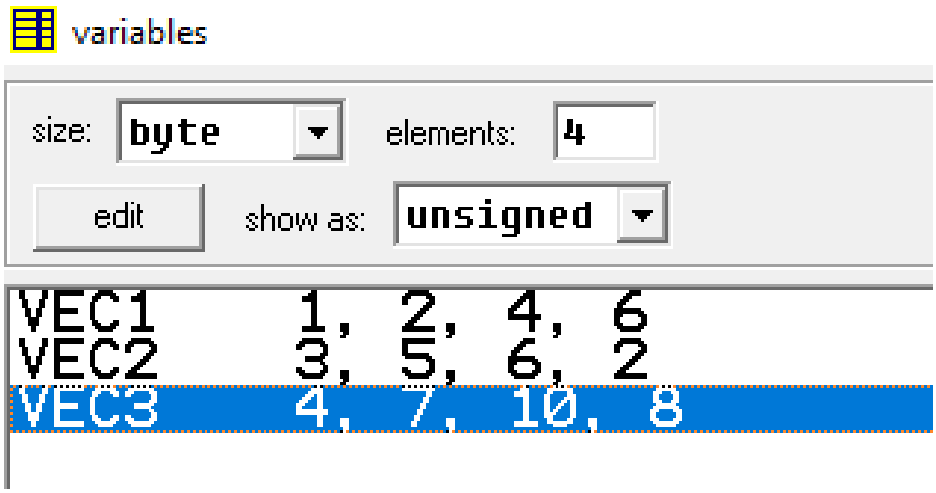
Procedure:

1. Open multisim software and create a new schematic.
2. Place 8 interactive digital constants, 7 half adders, 3 OR gate and ground on the schematic.
3. Connect the circuit as per circuit diagram.
4. Verify the circuit for all combinations.

Part-B: Assembly language program for adding to two arrays

```
org 100h
jmp start
vec1 db 1,2,4,6
vec2 db 3,5,6,2
vec3 db ?,?,?,?
start:
lea si,vec1
lea bx,vec2
lea di,vec3
mov cx,4
sum:
mov al,[si]
add al,[bx]
mov [di],al
inc si
inc bx
inc di
loop sum
ret
```

Output:



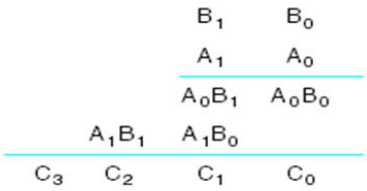
LAB-4

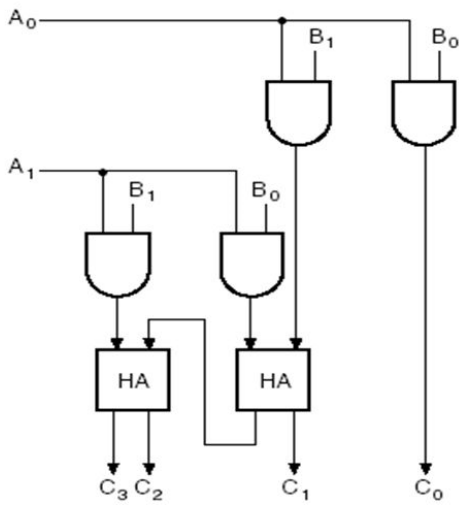
Part-A: 2x2 ARRAY MULTIPLIER

Aim: To construct and verify 2X2 array multiplier.

Description: An array multiplier is a digital combinational circuit used for multiplying two binary numbers by employing an array of full adders and half adders. This array is used for the nearly simultaneous addition of the various product terms involved. To form the various product terms, an array of AND gates is used before the Adder array.

Circuit Diagram:





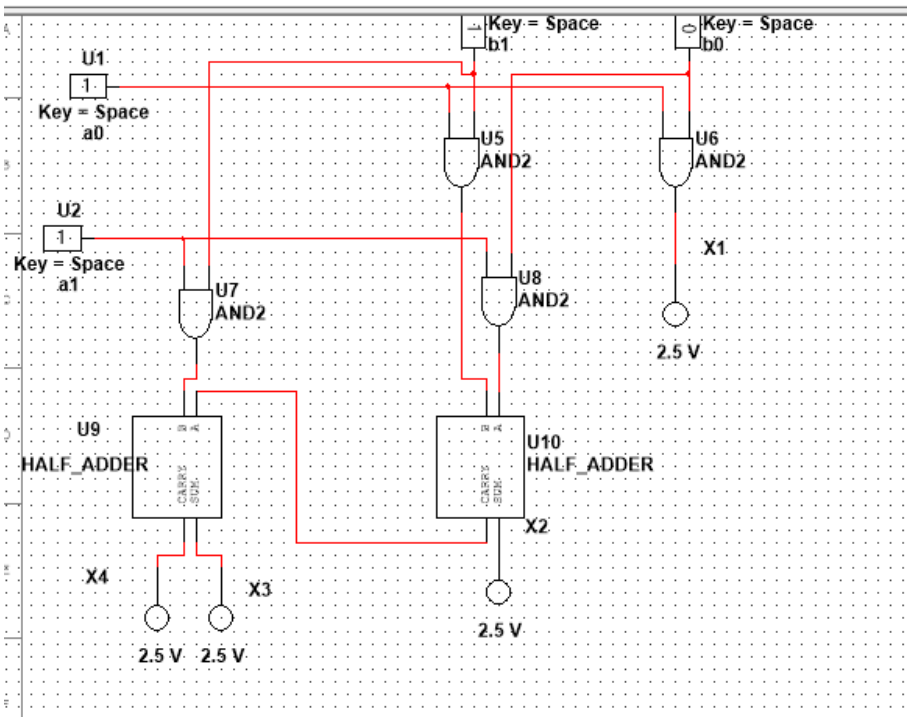
Truth Table:

A1	A0	B1	B0	S3	S2	S1	S0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1

Components required:

- Interactive digital constants
- Half adders
- AND gates
- Probes

Multisim circuit:



Procedure:

1. Open multisim software and create a new schematic.
2. Place the components on the schematic.
3. Connect the circuit as per circuit diagram.
4. Verify the circuit for all combinations.

Part-B:

Aim: Write an assembly language program to separate even and odd numbers from an array.

Input code:

```
ORG 100H
.MODEL SMALL
.STACK 100H
.DATA
ARRAY DB 1,2,3,4,5,6,7,8,9,10
EVEN_NUMS DB 10 DUP(?)
ODD_NUMS DB 10 DUP(?)
.CODE
MAIN PROC
 MOV AX, @DATA
 MOV DS, AX
 MOV CX,10
 MOV SI,0
 MOV DI,0
 MOV BX,0
 SEPERATE_LOOP:
 MOV AL, ARRAY[SI]
 TEST AL,1
 JZ IS_EVEN
 MOV ODD_NUMS[BX], AL;
 INC BX;
 JMP NEXT_ITERATION;
 IS_EVEN:
 MOV EVEN_NUMS[DI], AL;
 INC DI;
 NEXT_ITERATION:
 INC SI;
 LOOP SEPERATE_LOOP;
```

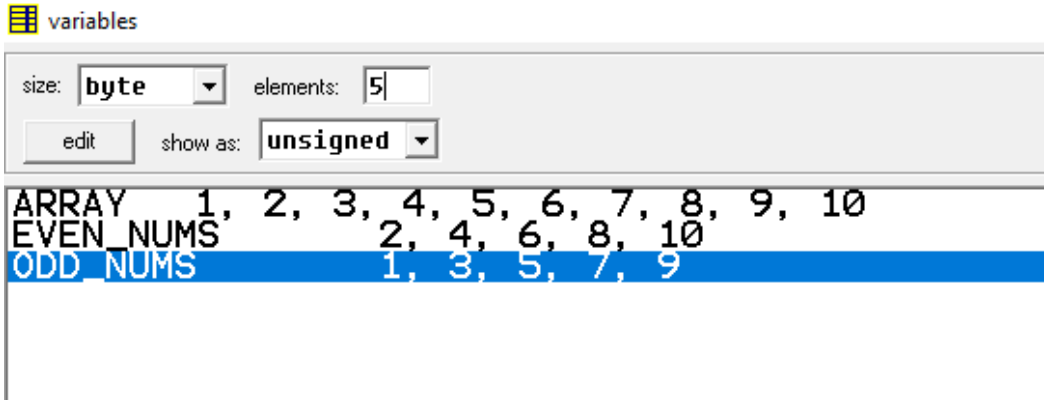
EXIT:

MAIN ENDP

END MAIN

RET

Output:



LAB-5

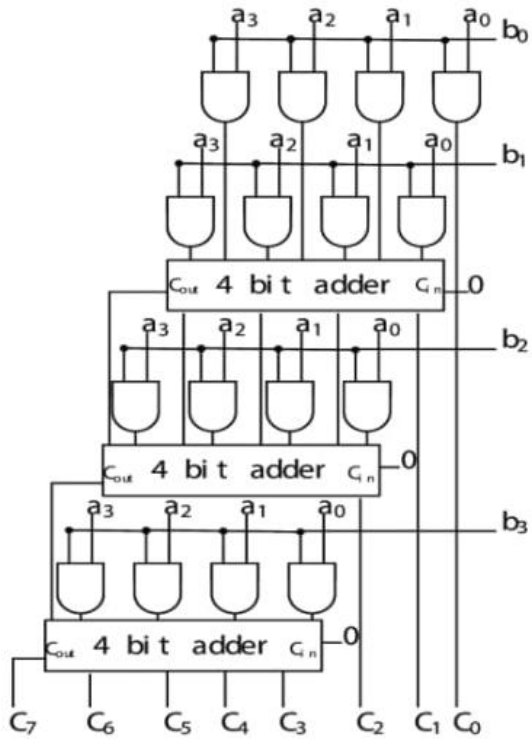
Part-A: 4X4 ARRAY MULTIPLIER

Aim: To construct and verify 4X4 array multiplier.

Description: Combinational Multiplier do multiplication of two unsigned binary numbers. Each bit of the multiplier is multiplied against the multiplicand, the product is aligned according to the position the bit within the multiplier, and the resulting products are then submitted to form the final result. Main advantage of binary multiplication is that the generation of intermediate products are simple, if the multiplier bit is a 1, the product is an approximately shifted copy of the multiplicand, if the multiplier bit is a 0, the product is simply a 0.

Circuit Diagram:

				a3	a2	a1	a0	Multiplicand
			x	b3	b2	b1	b0	Multiplier
				a3b0	a2b0	a1b0	a0b0	Partial Products
		+	a3b1	a2b1	a1b1	a0b1		
	+	a3b2	a2b2	a1b2	a0b2			
+	a3b3	a2b3	a1b3	a0b3				
P7	p6	p5	p4	p3	p2	p1	p0	Product



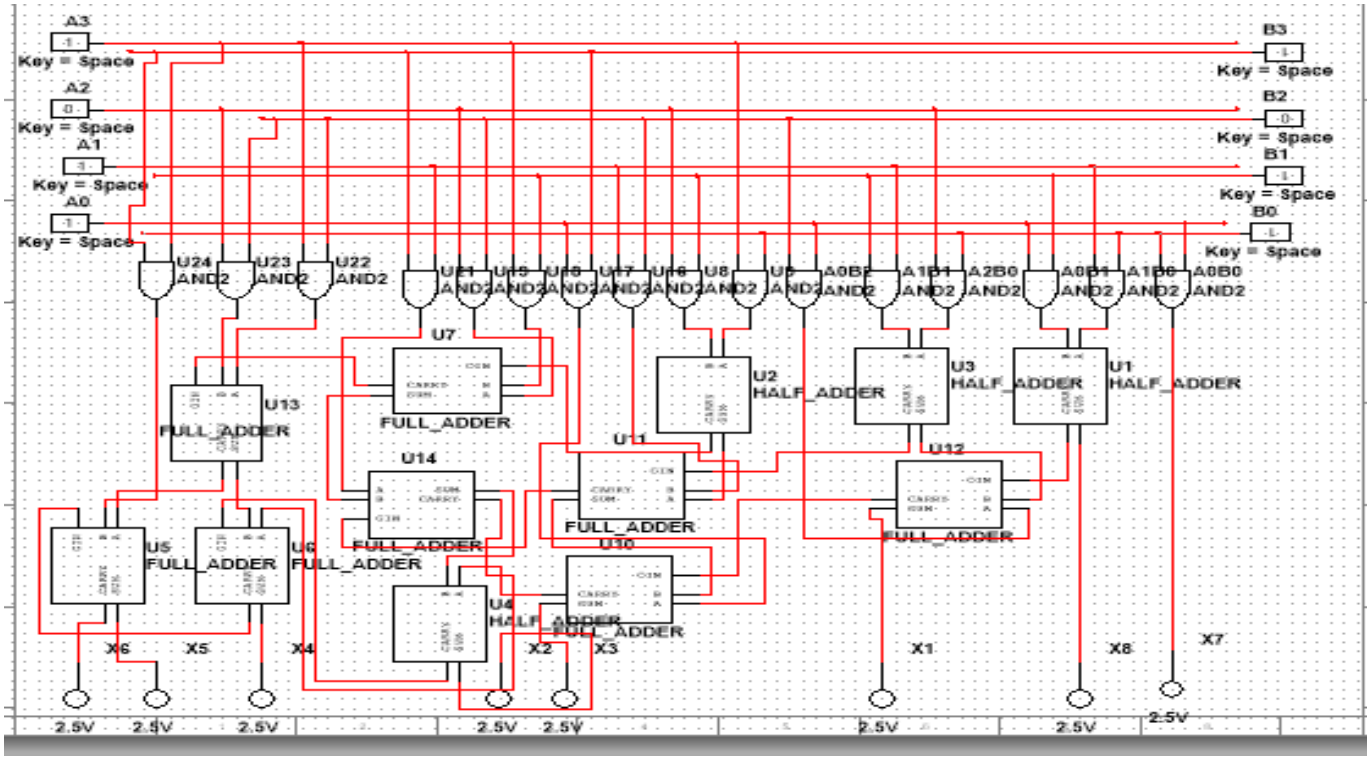
Truth Table:

A3	A2	A1	A0	B3	B2	B1	B0	S7	S6	S5	S4	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1
0	0	1	0	0	0	1	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	1	1	0	0	0	0	1	0	0	1
0	1	0	0	0	1	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	1	0	1	0	0	0	1	1	0	0	1
0	1	1	0	0	1	1	0	0	0	1	0	0	1	0	0
0	1	1	1	0	1	1	1	0	0	1	1	0	0	0	1
1	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0
1	0	0	1	1	0	0	1	0	1	0	1	0	0	0	1
1	0	1	0	1	0	1	0	0	1	1	0	0	1	0	0
1	0	1	1	1	0	1	1	0	1	1	1	1	0	0	1
1	1	0	0	1	1	0	0	1	0	0	1	0	0	0	0
1	1	0	1	1	1	0	1	1	0	1	0	1	0	0	1
1	1	1	0	1	1	1	0	1	1	0	0	0	1	0	0
1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	1

Components required:

- Interactive digital constant
- Half Adders and Full Adders
- AND gates
- Probes

Multisim Circuit:



Procedure:

1. Open Multisim Software and create a new schematic.
2. Place the components on the Schematic.
3. Connect the components as per circuit diagram.
4. Verify the circuit for all combinations.

Part-B:

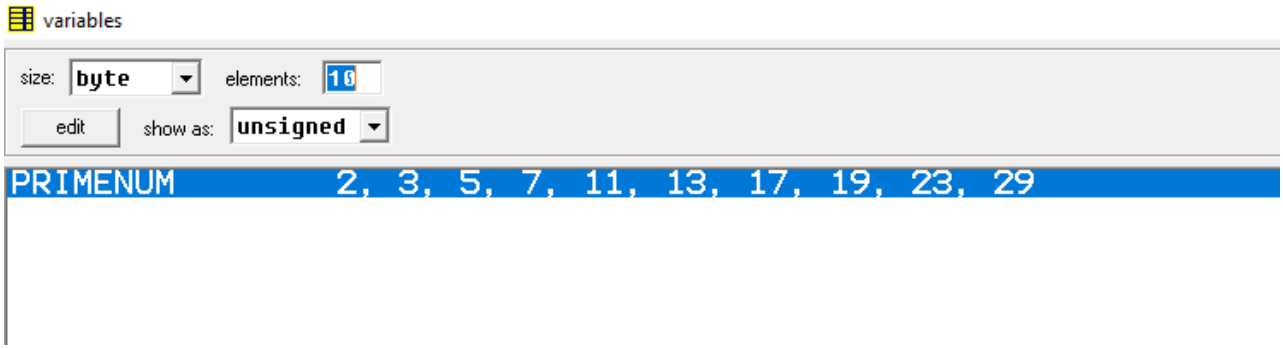
Aim: Write an assembly program to find all the prime numbers in the given range.

Input code:

```
org 100h
.model small
.stack
.code
main proc
    mov ax,@data
    mov ds,ax
    mov dl,1
    mov cx,10
    mov si,offset primenum
    l1:mov bl,2
    add dl,1
    cmp dl,2
    je insert
    logic:mov ah,0
    mov al,dl
    div bl
    cmp ah,0
    je l1
    add bl,1
    cmp bl,al
    jb logic
    jmp insert
    insert:mov [si],dl
    inc si
    loop l1
    mov ah,4ch
```

```
    int 21h
    main endp
.data
primenum db 10 dup(?)
end
ret
```

Output:



LAB-6

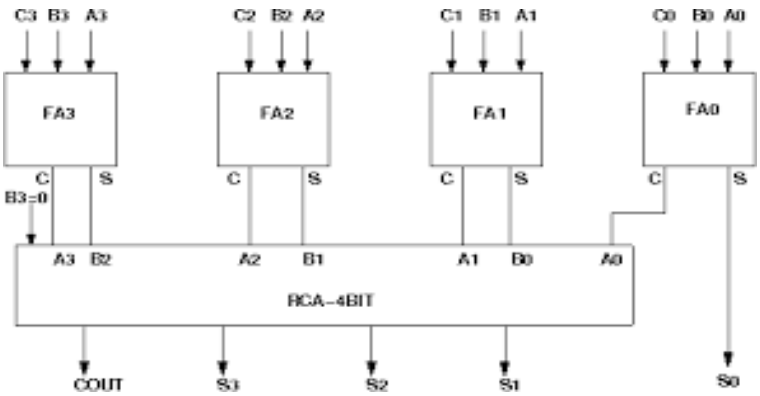
Part-A:WALLACE TREE ADDER

Aim: Verification of Wallace Tree Adder.

Description:

A Wallace Tree Adder is a hardware implementation of a binary numbers to add more than two binary numbers. To decrease the number of additions to solve the problem we will use Wallace tree adder. Here we use Half adders and Full adders to add the numbers.

Circuit Diagram:



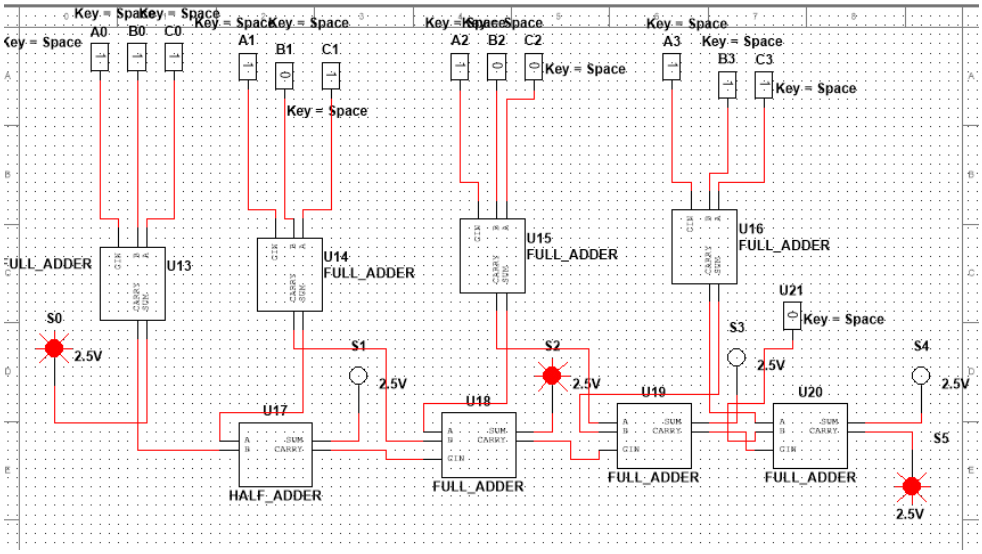
Truth table:

A3	A2	A1	A0	B3	B2	B1	B0	C3	C2	C1	C0	S5	S4	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1
0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	1	1	1
0	1	0	1	1	0	1	0	0	1	0	1	0	1	0	1	0	0
0	1	1	1	0	1	1	1	0	1	1	1	0	1	0	1	0	1
1	0	1	0	1	0	1	0	1	0	1	0	0	1	1	1	1	0
1	1	0	0	1	1	0	0	1	1	0	1	1	0	0	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0	1

Components Required:

- Interactive digital constants
- Full Adders and Half Adders
- Probes

Multisim Circuit:



Procedure:

1. Open multisim software and create a new schematic.
2. Place the components on the schematic.
3. Connect the circuit as per circuit diagram.
4. Verify the circuit for all combinations.

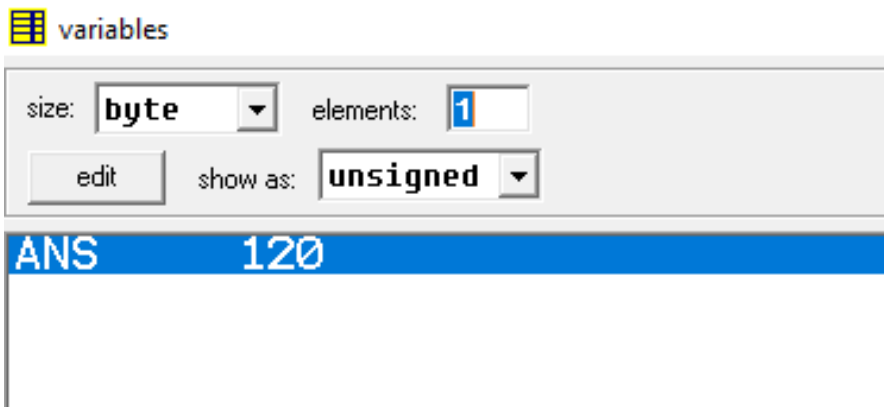
Part-B:

Aim: Write an assembly language program to find the factorial of a number.

Input code:

```
ORG 100H
.DATA
ANS DB ?
.CODE
MAIN PROC
 MOV AX, @DATA
 MOV DS, AX
 MOV AL,5
 MOV CL,4
 MOV BL, AL
 SUB BL, AL
 SUB BL,1
 L:
 MUL BL
 SUB BL,1
 LOOP L
 MOV ANS, AL
 END MAIN
RET
```

Output:



LAB-7

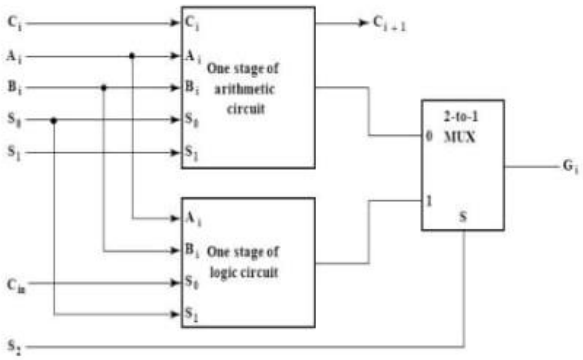
Part-A: ARITHMETIC LOGIC CIRCUIT

Aim: To construct arithmetic logic circuit.

Description:

Arithmetic logic unit is implemented by combining the Arithmetic unit and logic unit with the help of 2 by 1 multiplexer.

Circuit Diagram:



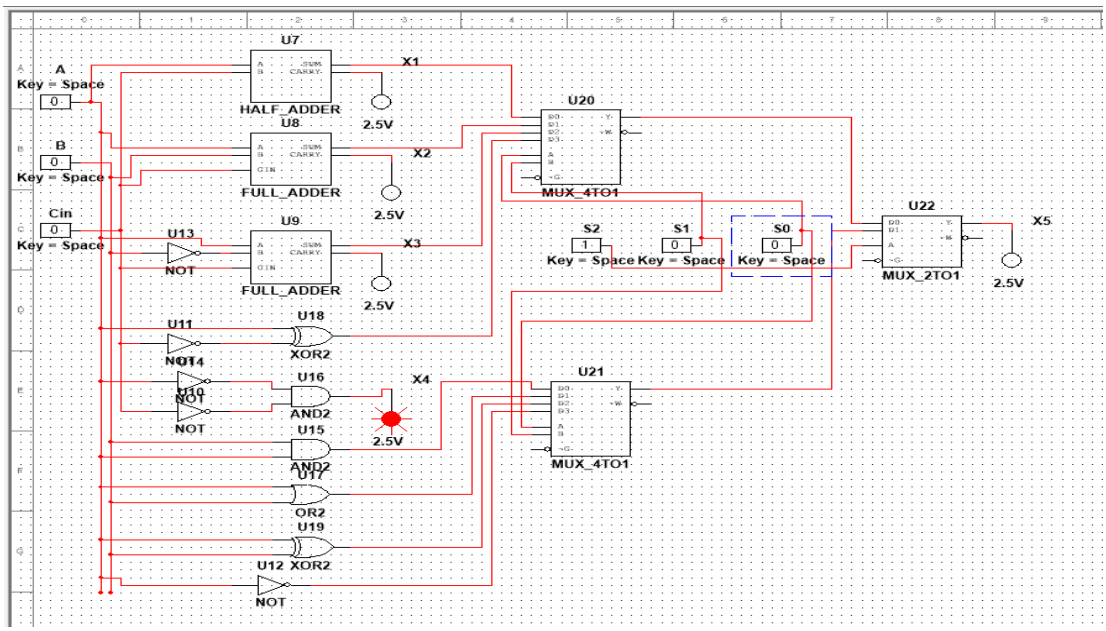
Truth Table:

S2	S1	S0	Cin	Operation	Function	A	B	Output
0	0	0	0	F=A	Transfer A	1	X	1
0	0	0	1	F=A+1	Increment A	0	X	1
0	0	1	0	F=A+B	Addition	1	1	0
0	0	1	1	F=A+B+1	Add with carry	0	1	0
0	1	0	0	F=A+B’	Subtract with borrow	1	1	1
0	1	0	1	F=A+B’+1	Subtraction	0	0	0
0	1	1	0	F=A-1	Decrement A	1	X	0
0	1	1	1	F=A	Transfer A	1	X	1
1	0	0	X	F=A^B	AND	1	1	1
1	0	1	X	F=A V B	OR	0	0	0
1	1	0	X	F=A XOR B	XOR	1	0	1
1	1	1	X	F=A’	Complement A	1	X	0

Components required:

- Interactive digital constant
- Half adders and Full adders
- AND,OR,NOT and XOR gates.
- Probe
- 4X1,2X1 mux

Multisim Circuit Diagram:



Procedure:

1. Open multisim software and create a new schematic.

2. Place the components on the required and connect them with the help of circuit diagram.
3. Give inputs to the arithmetic unit by using full adders and half adders and to gates by using input key.
4. Connect probes to the output lines of the multiplexer and wherever it is needed.
5. Run the schematic and check the implementation of ALU by varying the inputs according to the truth table.

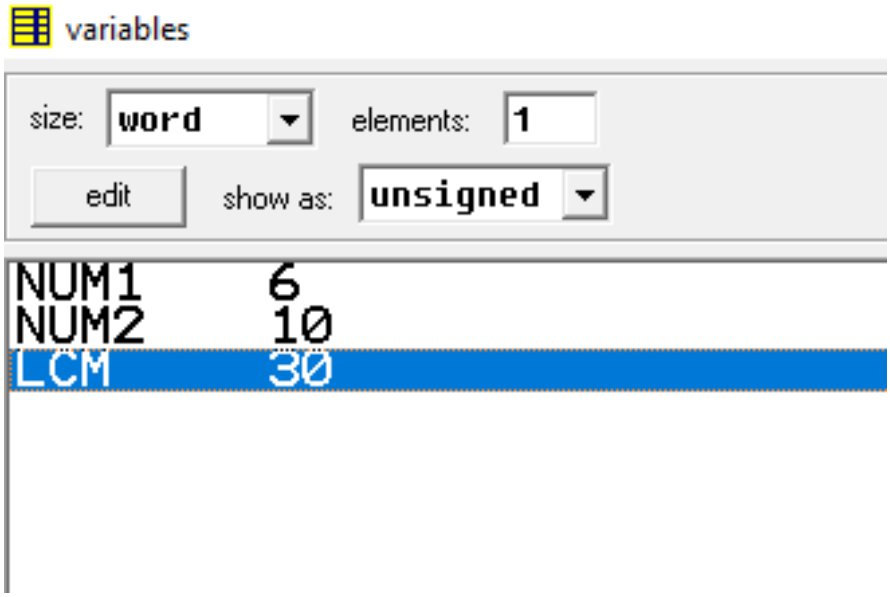
Part-B:

Aim: Write an Assembly Language program to find LCM.

Input code:

```
DATA SEGMENT
   NUM1 DW 6
   NUM2 DW 10
   LCM DW 2 DUP(?)
   DATA ENDS
CODE SEGMENT
   ASSUME CS:CODE,DS:DATA
   START:MOV AX,DATA
   MOV DS,AX
   MOV DX,0H
   MOV AX,NUM1
   MOV BX,NUM2
   UP:PUSH AX
   PUSH DX
   DIV BX
   CMP DX,0
   JE EXIT
   POP DX
   POP AX
   ADD AX,NUM1
   JNC DOWN
   INC DX
   DOWN:JMP UP
   EXIT:POP LCM+2
   POP LCM
   MOV AH,4CH
  CODE ENDS
END START
```

Output:


variables
size:
word
elements:
1
edit
show as:
unsigned
NUM1 6
NUM2 10
LCM 30

LAB-8

Part-A: REGISTERS AND COUNTERS

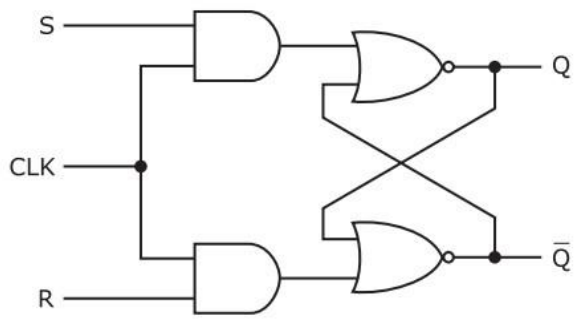
Part-1: Flipflops

Aim: Construction and verification of flipflops.

SR Flipflop:

Description: A Set-Reset (SR) flip-flop is a fundamental digital electronic circuit element used in digital systems for storing binary information. It's a type of sequential logic circuit, meaning its output depends not only on the present input but also on past inputs.

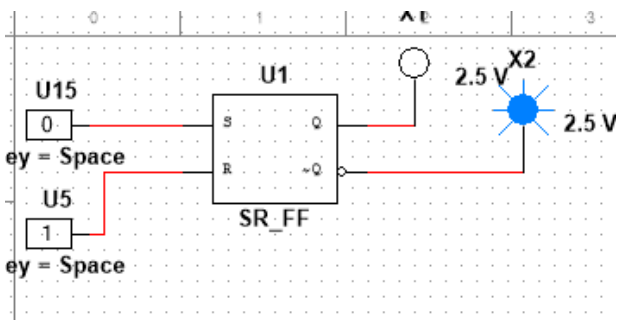
Circuit Diagram:



Truth table:

S	R	C	Q_{+1}	STATE
0	0	0	X	INVALID
0	0	1	X	
0	1	0	1	SET
0	1	1	1	
1	0	0	0	RESET
1	0	1	0	
1	1	0	0	NO CHANGE
1	1	1	1	

Multisim Circuit:



Components required:

- Probes
- Interactive digital constants
- SR flipflop

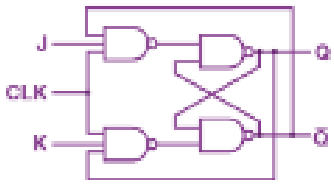
Procedure:

1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the truth table.

JK Flipflop:

Description: JK flip-flops are popular in digital design due to their versatility and reliability, especially when precise toggling functionality is required.

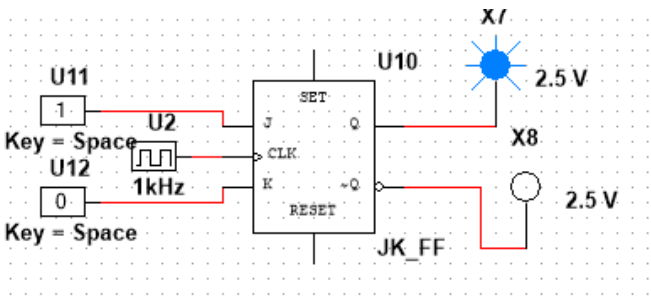
Circuit Diagram:



Truth Table:

J	K	Q_n	Q_{n+1}	State
0	0	0	0	Q_n (Hold)
0	0	1	1	
0	1	0	0	Reset
0	1	1	0	
1	0	0	1	Set
1	0	1	1	
1	1	0	1	Toggel
1	1	1	0	

Multisim Circuit:



Components required:

- Probes
- Interactive Digital Constants
- JK flipflop
- Clock

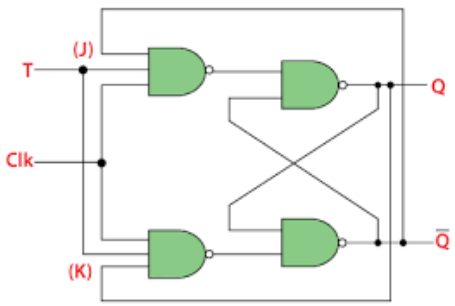
Procedure:

1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the circuit.

T Flipflop:

Description: A T flip-flop, also known as a Toggle flip-flop, is a type of digital sequential logic circuit that toggles its output state based on its input and current state.

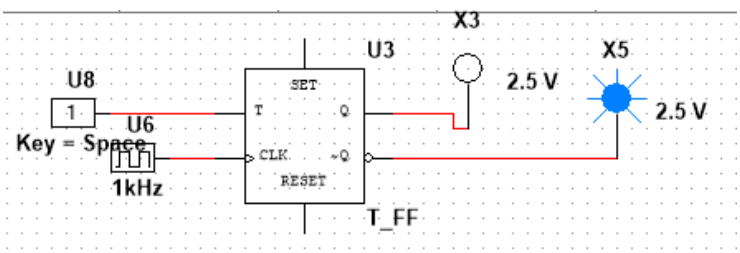
Circuit Diagram:



Truth Table:

T	Q	Q(t+1)
0	0	0
1	0	1
0	1	1
1	1	0

Multisim Circuit:



Components Required:

- Probes
- Interactive digital constants
- T Flipflop
- Clock

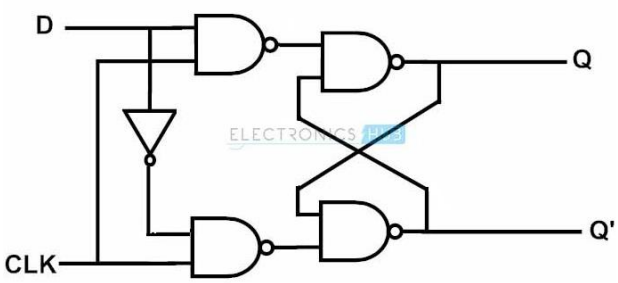
Procedure:

1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the circuit.

D Flipflop:

Description: A D flip-flop, also known as a Data flip-flop, is a type of digital sequential logic circuit that stores and outputs the state of its D (data) input when triggered by a clock signal.

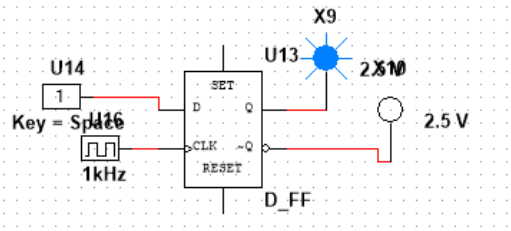
Circuit Diagram:



Truth Table:

D	Q(Current)	Q(n+1) (Next)
0	0	0
0	1	0
1	0	1
1	1	1

Multisim Circuit:



Components required:

- Probes
- Interactive digital constant
- D flipflop
- Clock

Procedure:

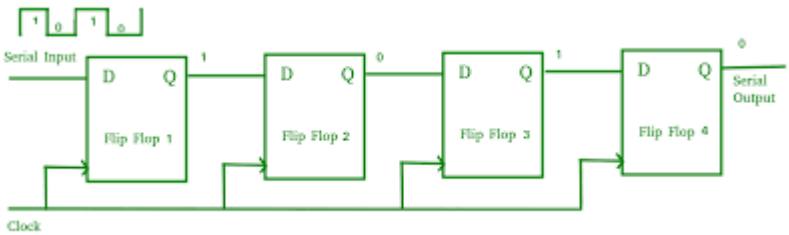
1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the truth table.

2.Registers:

1. 4 bit serial in serial out register:

Description: A 4-bit serial-in, serial-out (SISO) register is a digital circuit that stores a sequence of 4 bits of data and shifts them out one at a time in a serial fashion.

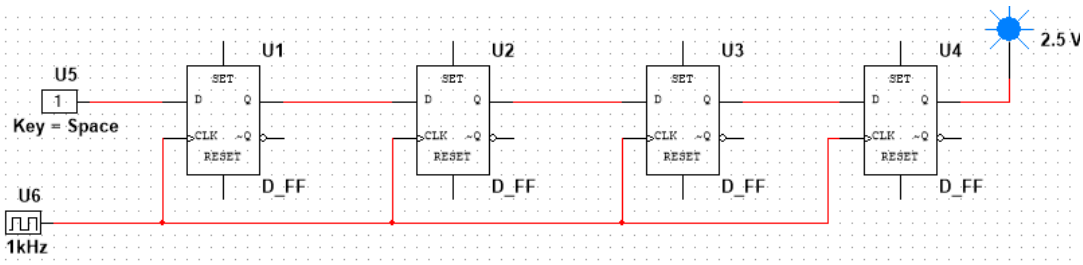
Circuit Diagram:



Truth Table:

input	QA	QB	QC	OUTPUT
1	1	1	1	1
0	0	0	0	0

Multisim Circuit:



Components Required:

- Probes
- Interactive digital constant
- D flipflop
- Clock

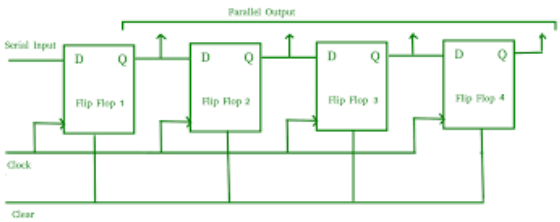
Procedure:

1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the truth table.

4 bit serial in parallel out register:

Desrciption: A 4-bit serial-in, parallel-out (SIPO) register is a digital circuit that accepts serial input data and stores it temporarily before outputting all bits in parallel simultaneously.

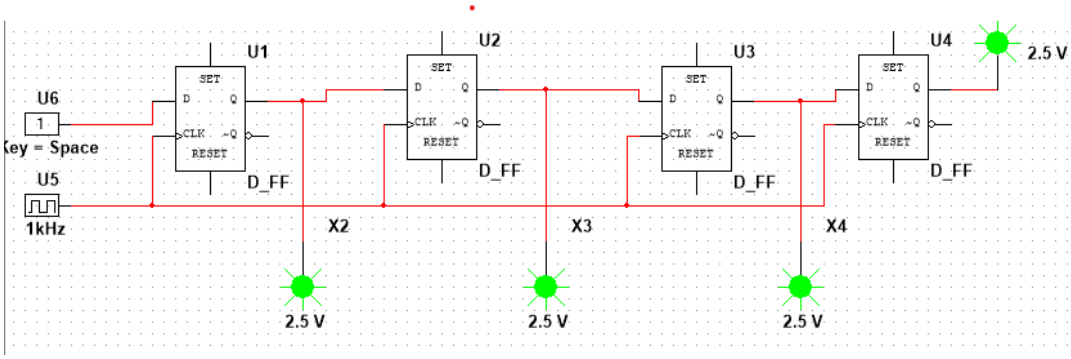
Circuit Diagram:



Truth Table:

INPUT	QA	QB	QC	QD
0	0	0	0	0
1	1	1	1	1

Multisim Circuit:



Components required:

- Interactive digital constants
- D flipflop
- Clock
- Probes

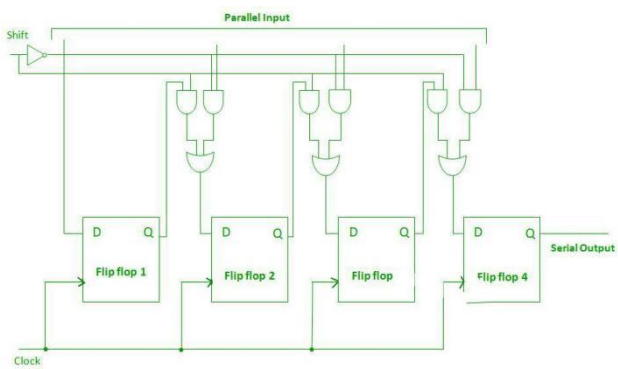
Procedure:

1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the truth table.

4 bit parallel in serial out register:

Description: A 4-bit parallel-in, serial-out (PISO) register is a digital circuit that accepts data in parallel format and outputs it serially.

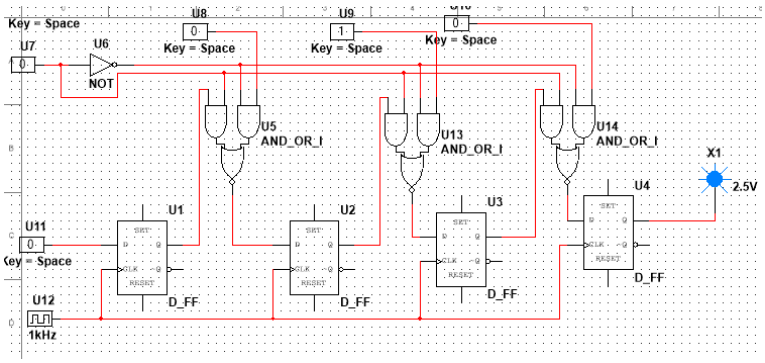
Circuit Diagram:



Truth Table:

QA	QB	QC	Output
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Multisim Circuit:



Components required:

- Interactive digital constant
- AND,OR,NOT gates
- D flipflop
- Probes

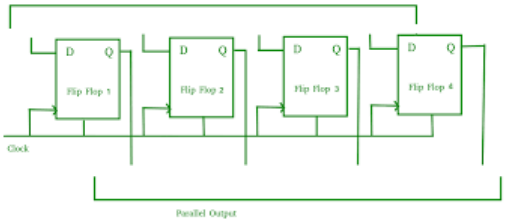
Procedure:

1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the truth table.

4 bit parallel in parallel out register:

Description: A 4-bit parallel-in, parallel-out (PIPO) register is a digital circuit that accepts data in parallel format and outputs it in parallel format without any conversion.

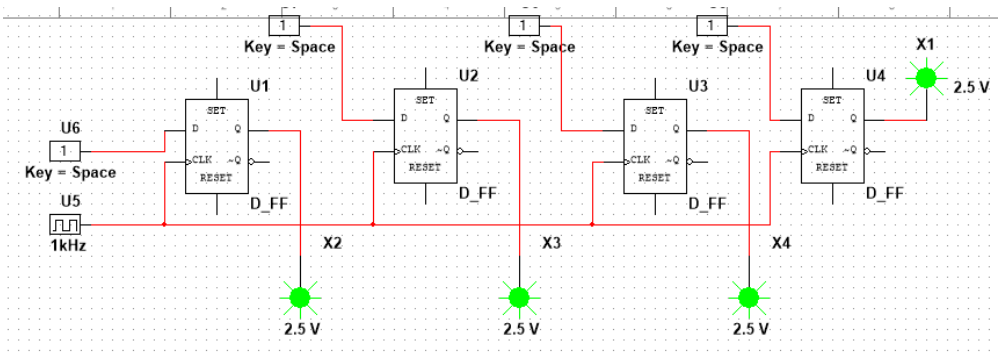
Circuit Diagram:



Truth Table:

A	B	C	D	Qa	Qb	Qc	Qd
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	1
1	0	1	0	1	0	1	0
1	0	1	1	1	0	1	1
1	1	0	0	1	1	0	0
1	1	0	1	1	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1

Multisim Circuit:



Components Required:

- Interactive digital constant
- JK flipflop
- Clock
- Probe

Procedure:

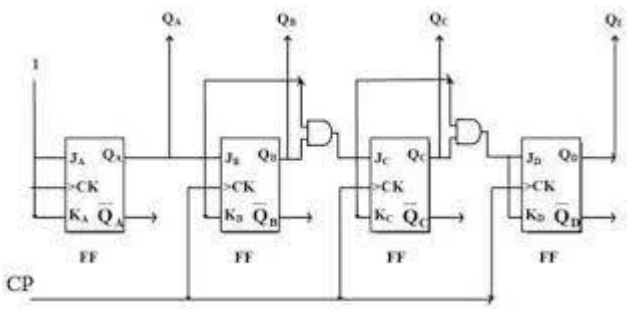
1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the circuit.

3.Counters

4 bit synchronous binary counter:

Description: A 4-bit synchronous binary counter is a digital circuit that counts in binary from 0 to 15 (2^4 - 1) and resets back to 0.

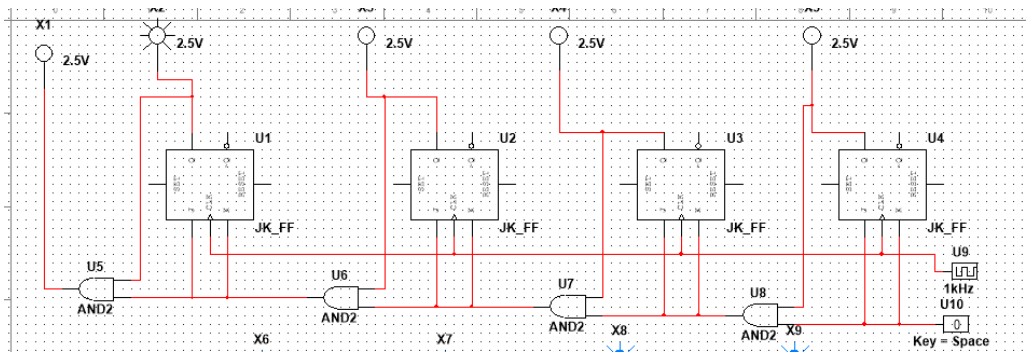
Circuit Diagram:



Truth Table:

CLK	Q4	Q3	Q2	Q1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1
16	0	0	0	0

Multisim Circuit:



Components required:

- Interactive digital constant
- JK flipflop
- AND gate
- Clock
- Probe

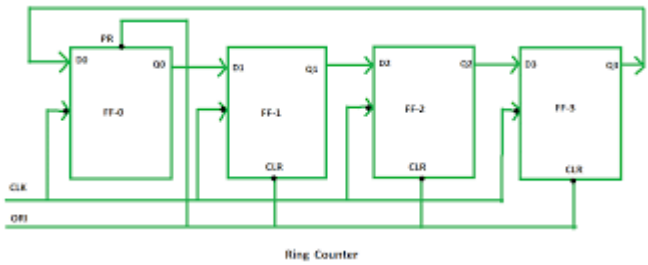
Procedure:

1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the circuit.

4 bit synchronous ring counter:

Description: A 4-bit synchronous ring counter is a type of digital counter circuit that produces a single active bit pattern that is shifted cyclically with each clock pulse.

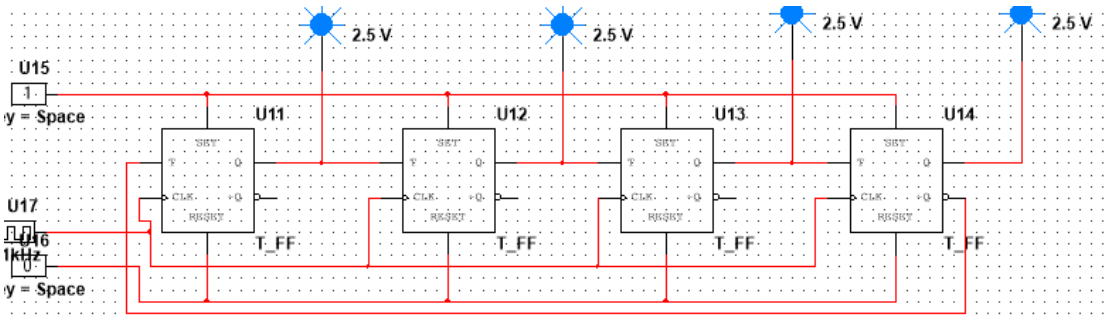
Circuit Diagram:



Truth table:

	CLK	Q0	Q1	Q2	Q3
	X	1	0	0	0
1	↓	0	1	0	0
1	↓	0	0	1	0
1	↓	0	0	0	1
1	↓	1	0	0	0

Multisim Circuit:



Components required:

- Interactive digital constants
- T flipflop
- Clock
- Probes

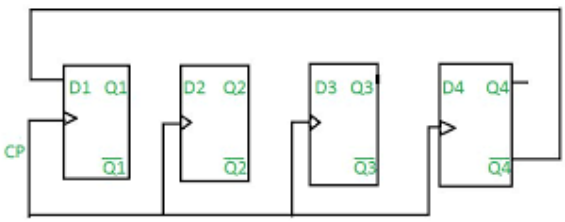
Procedure:

1. Open Multisim software and create a new schematic.
2. Connect the components as per circuit diagram.
3. Verify the circuit.

4 bit Jhonson Counter:

Description: A 4-bit synchronous Johnson counter, also known as a 4-bit synchronous twisted-ring counter, is a digital counter circuit that produces a specific sequence of states. It's similar to a ring counter but with a unique sequence.

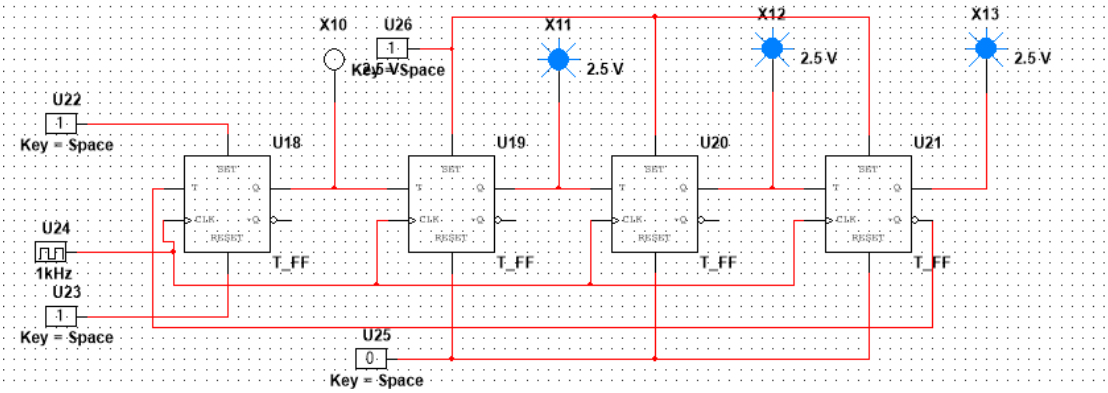
Circuit Diagram:



Truth Table:

CLR	CLK	Q0	Q1	Q2	Q3
	X	**0**	**0**	**0**	**0**
1	↓	1	0	0	0
1	↓	1	1	0	0
1	↓	1	1	1	0
1	↓	1	1	1	1
1	↓	0	1	1	1
1	↓	0	0	1	1
1	↓	0	0	0	1
1	↓	0	0	0	0
1	↓	1	0	0	0

Multisim Circuit:



Components required:

- Interactive digital constant
- Clock
- T flipflop
- Probes

Procedure:

1. Open Multisim software and create a new schematic.
2. Place the components on the schematic.
3. Connect the components as per circuit diagram.
4. Verify the circuit with truth table.

Part-B:

Aim: Write an Assembly Language program to find GCD.

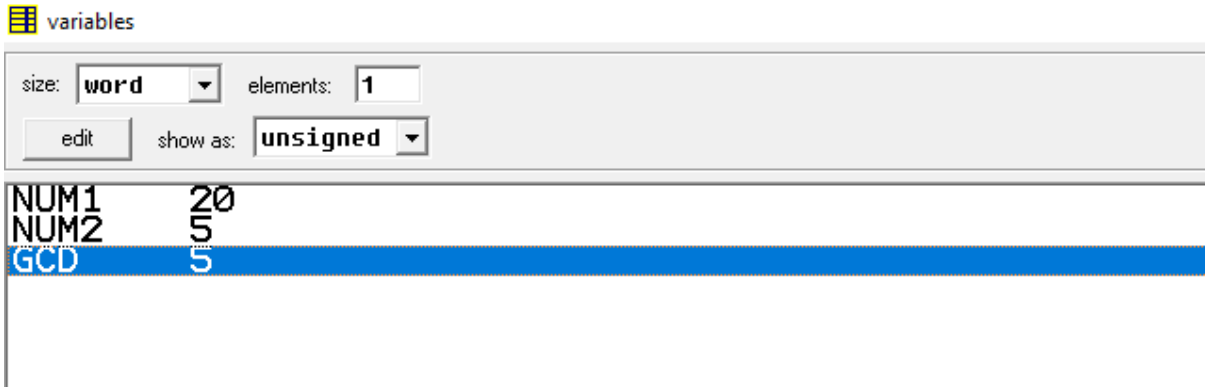
Input code:

```
DATA SEGMENT
   NUM1 DW 20
   NUM2 DW 5
   GCD DW ?
   DATA ENDS
CODE SEGMENT
   ASSUME CS:CODE,DS:DATA
   START:
   MOV AX,DATA
   MOV DS,AX
   MOV AX,NUM1
   MOV BX,NUM2

   UP:
   CMP AX,BX
   JE EXIT
   JB EXCG
   UP1:
   MOV DX,0H
   DIV BX
   CMP DX,0
   JE EXIT
   MOV AX,DX
   JMP UP
   EXCG:
   XCHG AX,BX
   JMP UP1
   EXIT
```

```
    MOV GCD,BX
    MOV AH,4CH
    INT 21H
    CODE ENDS
END START
```

Output:



Lab-9

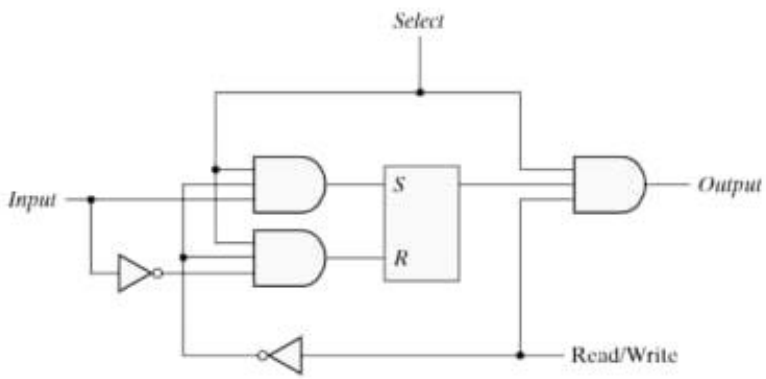
Part-A: VERIFICATION OF 1-BIT RAM CELL AND 4X4 RAM CELL

Aim: To design and verify 1 bit RAM cell and 4X4 RAM cell.

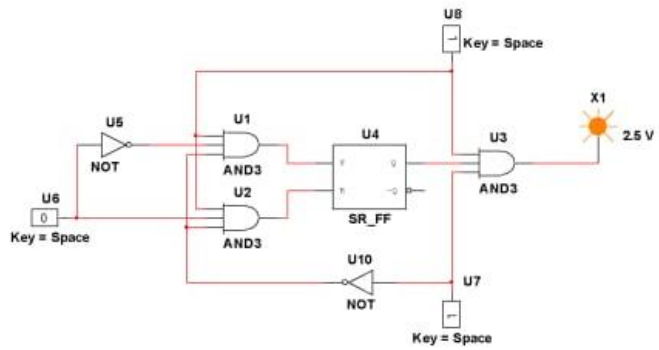
1-BIT RAM CELL:

Description: A 1-bit RAM cell, short for Random Access Memory cell, is the fundamental building block of memory in digital electronic devices. It stores a single bit of data, which can be either a 0 or a 1. The basic structure of a 1-bit RAM cell typically involves a combination of logic gates, such as transistors, capacitors, or flip flops.

Circuit Diagram:



Multisim Circuit:



Components required:

- SR flipflop
- 3-input AND gates,NOT gates
- Probe
- Interactive Digital constants

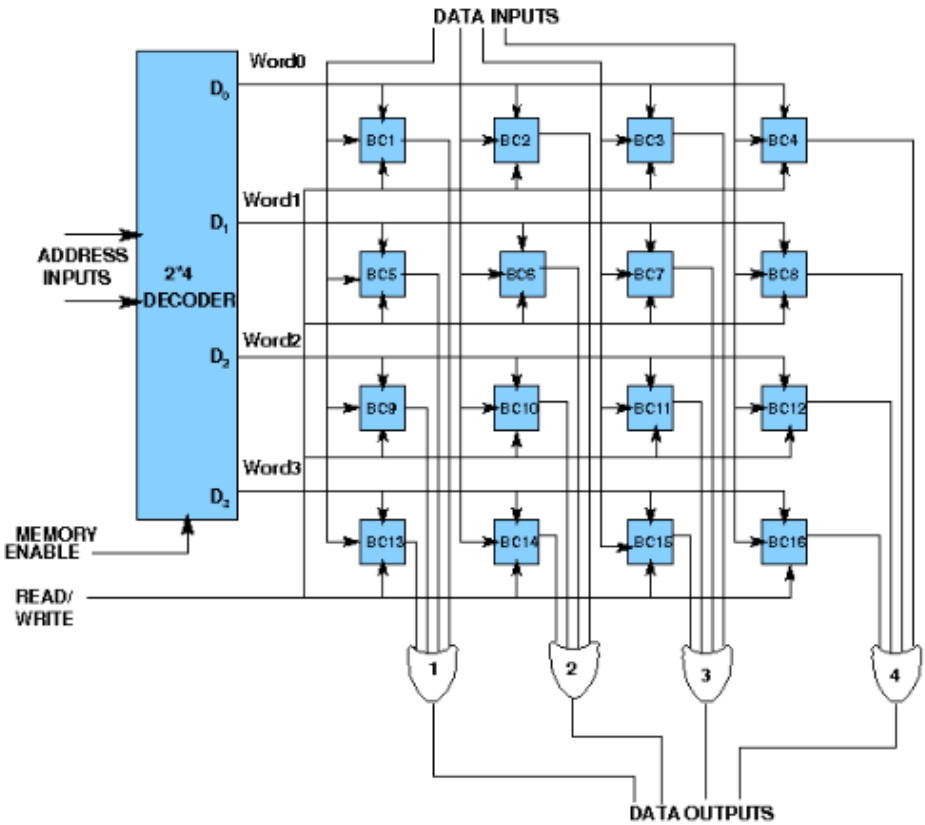
Procedure:

1. Open Multisim software and create a new schematic.
2. Place the components on the schematic.
3. Connect the components as per circuit diagram.
4. Verify the circuit.

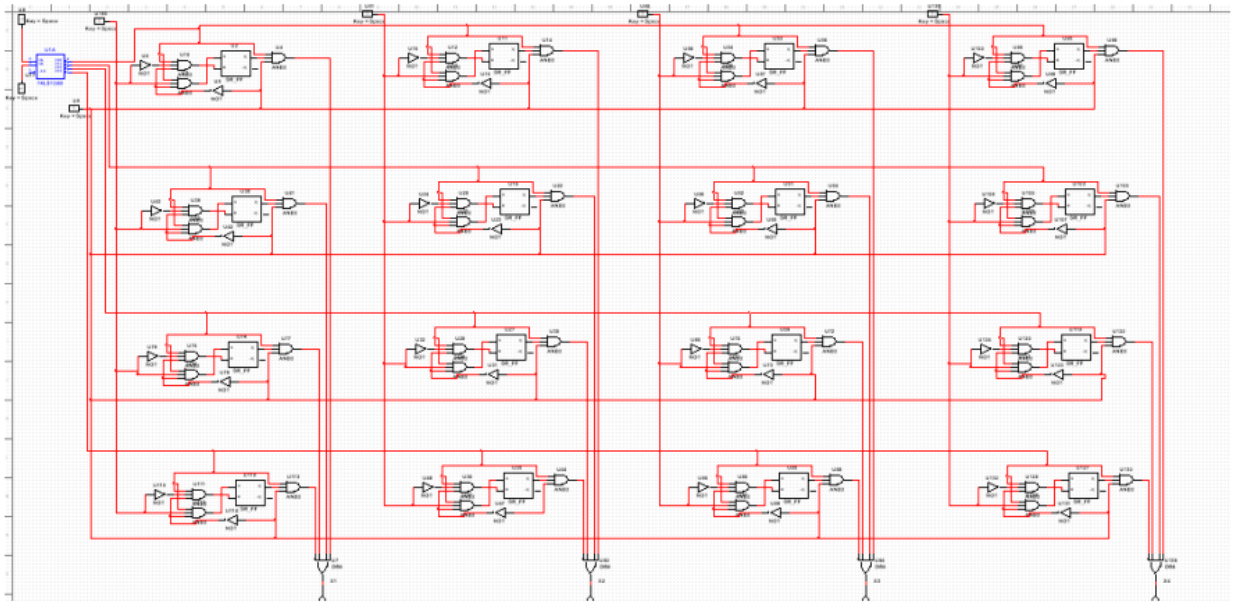
4X4 RAM Cell:

Description: A 4x4 RAM cell, or 4-bit by 4-bit Random Access Memory cell, is a fundamental unit within computer memory architecture used for storing and retrieving digital information. It is commonly found in integrated circuits and forms the building block for larger memory structures.

Circuit Diagram:



Multisim Circuit:



Components Required:

- NOT, AND gates
- SR flipflops
- Probes
- Interactive Digital constants
- Decoder.

Procedure:

1. Open Multisim software and create a new schematic.
2. Place the components on the schematic.
3. Connect the components as per circuit daiagram.
4. Verify the circuit.

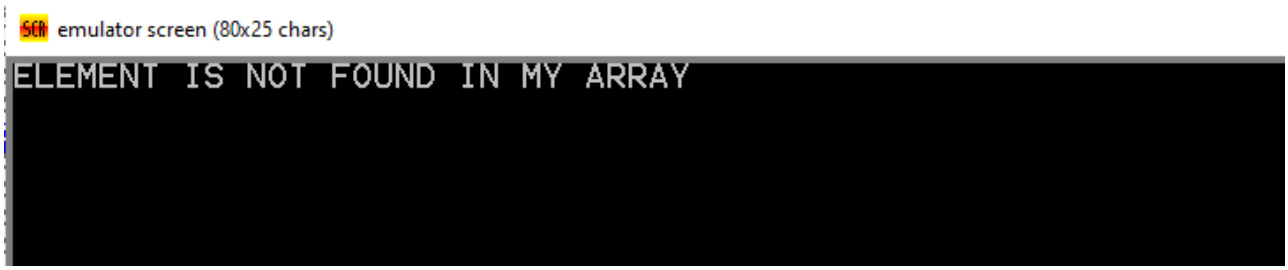
Part-B:

Aim: Write an Assembly Language program to search an element using linear search.

Input code:

```
.MODEL SMALL
.DATA
NUM DB 11,44,65,5,30
MSG1 DB "ELEMENT IS FOUND IN MY ARRAY $"
MSG2 DB "ELEMENT IS NOT FOUND IN MY ARRAY $"
KEY DB 4
PRINT MACRO MSG
 MOV AH,09H
 LEA DX, MSG
 INT 21H
ENDM
.CODE
MOV AX, @DATA
MOV DS, AX
LEA SI, NUM
MOV CX,5
MOV AL, KEY
HERE:
MOV BL, [SI]
CMP AL, BL
JZ FOUND
INC SI
LOOP HERE
PRINT MSG2
JMP EXIT
FOUND:
PRINT MSG1
EXIT:
MOV AH,4CH
```

INT 21H

.EXIT

Output: